When NUMERIC(18) Isn't Enough

A Vertica Field Guide for Eight Real Use Cases *May 2026*

Moshe Goldberg

Abstract

Eight numeric design questions sit at the heart of getting the most out of a Vertica deployment. Some are familiar ("how do I size my SUM headroom?"), some are subtle ("how does projection sort order shape NUMERIC storage?"), and a few are quietly load-bearing for production workloads. This field guide walks through each one with measured evidence from a five-node Vertica v26.1 cluster running benchmarks at 500 million and two billion rows, with every claim grounded in numbers the engine produced in front of us.

The reward for the time you invest reading this is a clear answer to each scenario, the smallest design changes that get you there, and an understanding of why. Several of the findings highlight Vertica's projection design as a first-class storage and performance lever - get it right and you save dramatically, take the defaults and you still get exact, predictable behavior.

The full article in eight quick answers (FAQ):

UC1: "My SUMs keep overflowing on NUMERIC(18). Do I need to migrate the column?"

No. Just cast inside the aggregate: SUM(v::NUMERIC(38,0)). The column stays in the fast 8-byte band, all your other queries keep INTEGER-class performance, and the SUM gets enough headroom for any realistic row count. Quick math: required precision = digits in your values + ceil(log10(row count)). If that exceeds 18, cast.

UC2: "I really do need 19+ digits, and I can sort the projection on this column. How bad is it?"

Not bad at all. Storage on high-entropy data is about 1.7× NUMERIC(18); on bounded data with the typed column as the sort key, AUTO encoding compresses it down to 0.15× — yes, six times *smaller* than NUMERIC(18). Runtime is within 1.1–1.3× of NUMERIC(18) for hashing operations. **Pick NUMERIC(28)** — it costs the same at runtime as NUMERIC(19), gives you nine extra digits of headroom for free, and avoids a second migration when your data grows.

UC3: "I need 19+ digits, but my projection is sorted by something else (timestamp, customer_id, etc.). What changes?"

The cost goes up, but it's still manageable. Plan for ~4× storage, 2.7× GROUP BY, and 3.2× JOIN. INSERT cost stays at ~1.3× regardless of sort order, which is reassuring for migration timing. The most effective fix: add a **dedicated projection** sorted on the typed column for queries that filter or aggregate by it, while keeping your main projection sorted by your dominant access pattern. One thing that doesn't help: putting the typed column second in a multi-column ORDER BY — AUTO encoding gives no partial credit for that.

UC4: "Should I just use FLOAT? It's faster, right?"

It depends entirely on your data, not on speed. Choose by exactness first, range second:

- Currency, IDs, counts, anything where equality must work → **NUMERIC**
- Measurements, ratios, statistics that fit in ~15 digits → **FLOAT** (genuinely faster on aggregations)
- Need exact 16+ digits → **NUMERIC(19+)**, no shortcut

FLOAT silently rounds at ~15–17 significant digits — that's IEEE 754, not a Vertica quirk. Use it where approximation is acceptable; avoid it where it isn't.

UC5: "I'm storing sensor data and measurements at scale. NUMERIC or FLOAT?"

FLOAT, with confidence. On high-entropy 18-digit data, FLOAT scans in **half the time** of NUMERIC(18) and runs GROUP BY at essentially the same speed. Storage overhead vs NUMERIC(18) is modest (6.6 vs 4.9 bytes/row). Sensor readings, telemetry, prices that don't need to balance to the cent — FLOAT's 15-digit precision is enough, and your downstream visualization tools probably consume FLOAT anyway.

UC6: "My AVG is silently rounding huge numbers. What do I do?"

Switch to manual SUM/COUNT and cast inside the SUM:

sql

```
SELECT SUM(v::NUMERIC(38,0)) / COUNT(*) AS exact_avg FROM big_table;
```

The cliff is at exactly 17 digits — built-in AVG() is exact through 16, lossy from 17 onward (because it returns FLOAT). The pattern above keeps NUMERIC arithmetic end-to-end and was exact at every magnitude we tested. For extreme precision-times-volume workloads, the exact_avg() UDx handles the tail.

UC7: "It's currency. Just tell me what to do."

NUMERIC, never FLOAT. Floating-point silent rounding is dangerous when pennies must balance.

- Most retail/financial values fit in **NUMERIC(18,2)** or **NUMERIC(18,0)** (storing in cents/satoshis) — INTEGER-class fast path.
- If aggregations might exceed 18 digits → use the UC1 pre-cast pattern, no migration needed.
- If individual values exceed 18 digits (large institutional transfers, certain crypto raw amounts) → **NUMERIC(28)** and follow UC2 or UC3 depending on projection design.

UC8: "How do I migrate an existing NUMERIC(18) column to wider precision?"

Use create-and-swap, not ALTER. Direct ALTER TABLE ... SET DATA TYPE fails across the precision band (ROLLBACK 2377). The in-place ADD/DROP/RENAME pattern fails on default-segmented k-safe tables (ROLLBACK 4122 — buddy projections). The pattern that works:

```
-- 1. Build replacement table with target type
CREATE TABLE schema.t_new (... col NUMERIC(28,0) ...);

-- 2. Copy with cast
INSERT /*+DIRECT*/ INTO schema.t_new
SELECT id, col::NUMERIC(28,0), ... FROM schema.t;
```

```
-- 3. Atomic rename (single metadata operation)
ALTER TABLE schema.t, schema.t_new RENAME TO t_old, t;

-- 4. Drop old after verification
DROP TABLE schema.t_old CASCADE;
```

Six steps - When you just need the recipe:

The shared mental model

Before the eight use cases, three mechanics tie everything together. Spending 90 seconds on these makes the rest of the article click.

1. The 8/16-byte boundary. Vertica stores NUMERIC values in fixed-width slots: precision 1–18 fits in 8 bytes, precision 19–37 in 16 bytes, 38–56 in 24 bytes, and so on in 8-byte steps up to NUMERIC(1024). Inside the 8-byte band Vertica documents that NUMERIC performance is equivalent to INTEGER. Crossing to 19 digits steps into the 16-byte band, which is no longer the INTEGER fast path - although as you'll see, the cost depends much more on projection design than on the precision itself.

2. AUTO encoding rewards good projection design. Every test in this guide uses Vertica's default AUTO encoding. AUTO is remarkably good when the projection is sorted by the typed column (long runs of equal or similar values compress aggressively). It is also fair when the column is unsorted (it pays roughly the slot width). What it doesn't do is interpolate between the two - putting the typed column second in a multi-column ORDER BY gives the same result as not sorting on it at all. Sort order, in other words, is binary for compression purposes.

3. Apples-to-apples is the only comparison. All numbers below come from the same source data, materialized once and cast into per-type tables; matched-type joins; matched-type insert sources; and ratios reported against the appropriate baseline (sorted-vs-sorted, or unsorted-vs-unsorted). Every storage and performance ratio in this article was produced by code we shared in GitHub.

Methodology, in one paragraph

Five Vertica 26.1 nodes, ~16 GB RAM each. Each typed table holds 2 billion (id, k, v) rows, with k as the tested type and projection ORDER BY k SEGMENTED BY HASH(k) ALL NODES, AUTO encoding. A second set of tables uses ORDER BY id to model the realistic case where the typed column isn't the primary sort key. Per-node row balance is within 0.122% across all eight typed tables, confirmed via projection_storage. Configuration parameters (AllowNumericOverflow, NumericSumExtraPrecisionDigits) are at defaults. Every query was profiled cold-cache after CLEAR_CACHES(); we report the median of trailing iterations after dropping the first as warm-up. Where storage is reported, it's the bytes/row of the typed column from column_storage.

Use Case 1 - "My SUMs are overflowing on NUMERIC(18)"

This is the use case that started the whole investigation. The column itself fits comfortably in 18 digits, but a SUM across many rows produces a result that does not. On a default-configured Vertica cluster, here is exactly what we observed.

100 million rows, each holding the value 999999999999999999 stored as NUMERIC(18,0). The mathematical answer is 99,999,999,999,999,999,900,000,000 - twenty-six digits. The built-in SUM(v) returns –2,537,764,290,215,403,776: a negative number, silently wrapped, no error raised. Pre-casting the column inside the aggregate - SUM(v::NUMERIC(38,0)) - returns the exact 26-digit result.

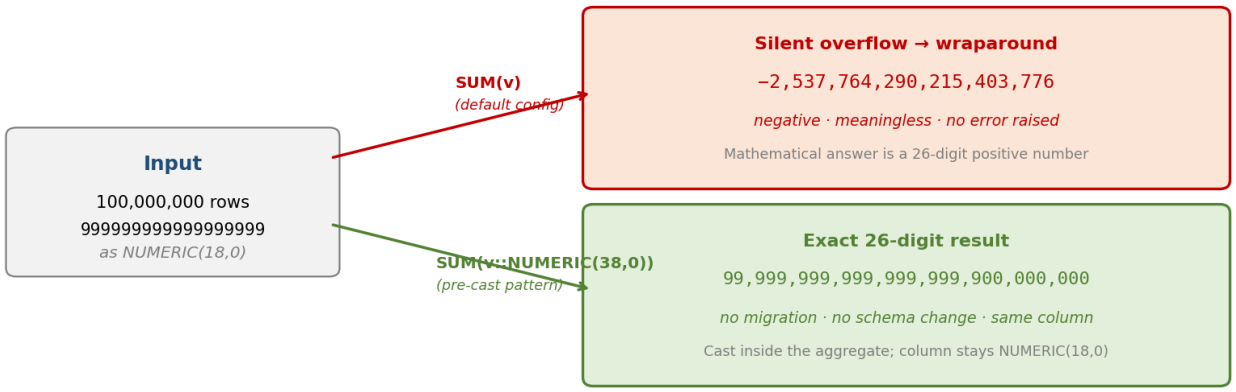

Figure 1 - Silent SUM overflow on NUMERIC(18) and the pre-cast fix that returns the exact 26-digit result.

The behavior is governed by configuration parameters, most prominently AllowNumericOverflow (default 1, which permits silent overflow) and NumericSumExtraPrecisionDigits (default 6, which adds a few digits of headroom that's enough for many but not all workloads). The cluster we tested has both at the documented defaults, and the 100M × 18-digit case still overflowed, so this isn't a corner case - it's something most Vertica deployments are exposed to without realizing.

Recommendation

If your column itself fits in 18 digits but SUM might exceed that, do not migrate the column. Cast inside the aggregate: SUM(v::NUMERIC(38,0)). The column stays in the 8-byte band, all other queries keep INTEGER-class performance, and the SUM gets enough headroom for any realistic row count. The rule of thumb: required precision = p_in + ceil(log10(N)). If that exceeds 18, cast - or set AllowNumericOverflow to 0 so silent wraparound becomes an error you can catch.

Use Case 2 - "I genuinely need 19+ digits, and the projection IS sorted on this column"

Some columns truly need more than 18 digits - large identifiers, deduplication keys, scientific values that exceed 10^18. When that column happens to be the natural sort key for the dominant access pattern (lookups by id, range scans by hash), you are in the favorable case. The cost of crossing the precision band is moderate, and inside the 19–37 range you can buy substantial headroom for free.

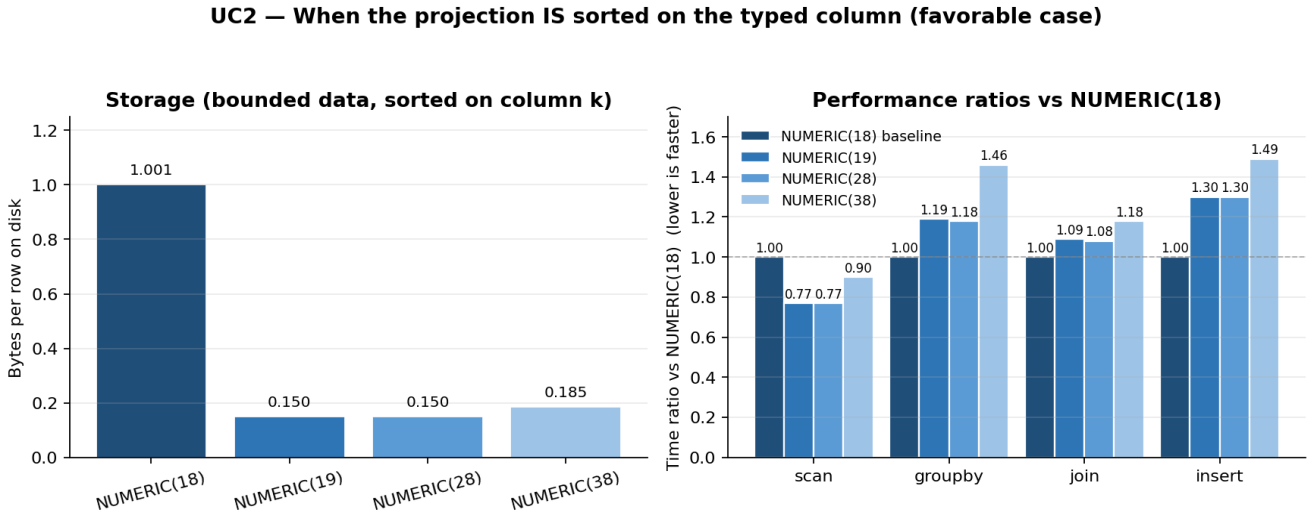

Figure 2 - Storage and performance with the projection sorted on the typed column. NUMERIC(28) is essentially identical to NUMERIC(19) at runtime; NUMERIC(38) costs 8–22% more.

Two findings from the figure deserve attention. First, scan is actually faster at NUMERIC(19) than at NUMERIC(18) on this favorable shape - not because arithmetic is cheaper (it isn't), but because AUTO encoding compresses the wider type's high-order bytes so aggressively that there are far fewer bytes to read from disk. The I/O savings overwhelm the wider arithmetic cost. Second, every precision from NUMERIC(19) through NUMERIC(28) costs the same at runtime - within 1% across scan, GROUP BY, and JOIN - and only NUMERIC(38) starts to add a measurable 8–22% on top. We confirmed this with an interleaved benchmark of n19, n28, and n38 in the same session, with cold caches between every query.

INSERT cost is the one place NUMERIC(19) lands meaningfully above NUMERIC(18) - about 1.30× - and that increment is genuinely the cost of writing 16-byte values instead of 8-byte ones. There's no projection trickery that erases this; if INSERT throughput is the bottleneck, it's a real consideration.

Recommendation

When the typed column needs to cross the band and you can sort the projection on it, pick the smallest precision in the next band that comfortably accommodates future growth. NUMERIC(28) is a sweet spot: it costs the same at runtime as NUMERIC(19), gives you nine extra digits of headroom, and avoids a second migration if your data grows. Use the create-and-swap pattern in Use Case 8 to perform the migration.

Use Case 3 - "I need 19+ digits, but the projection is sorted on something else"

This is the realistic majority of production fact tables. The projection's primary sort key is timestamp, customer_id, transaction_date, or some other dimension that suits the dominant query pattern. The numeric column you're widening is incidental to the sort. This is the case where the v07 of this article quietly under-reported the cost - the dramatic compression numbers we showed previously were measured with the typed column as the primary sort. When that's not how your projection is laid out, the picture changes substantially.

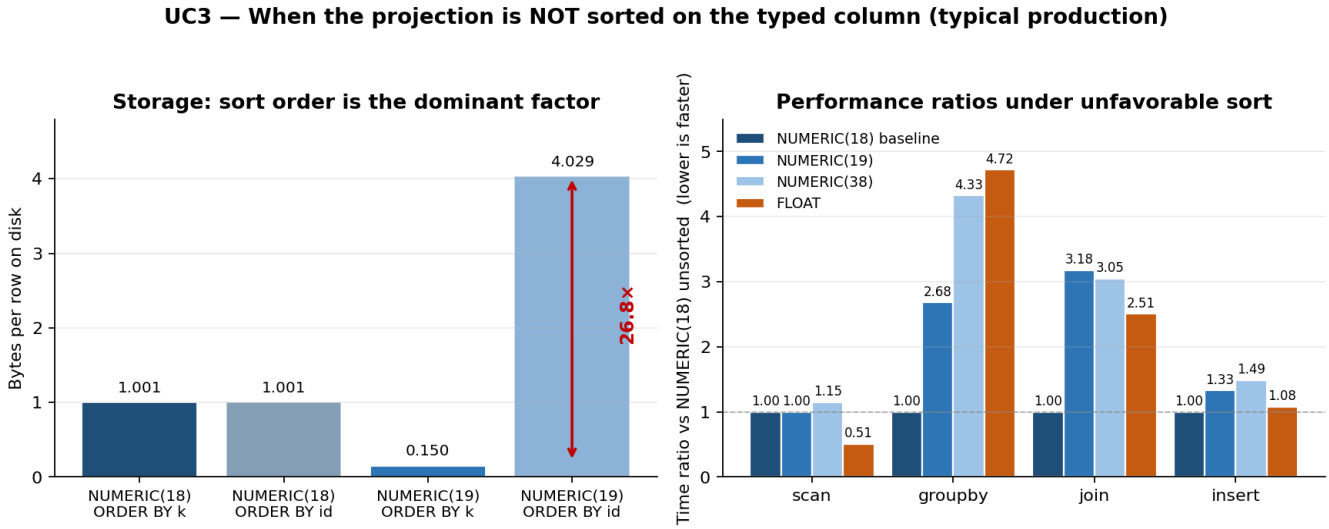

Figure 3 - Sort order is the dominant storage and performance factor when the typed column isn't the primary sort key. Storage scale is bytes/row; performance is ratios versus NUMERIC(18) under the same unfavorable sort.

Storage first. With the typed column as primary sort, NUMERIC(19) on bounded data sits at 0.15 bytes/row - AUTO compresses the constant high-order bytes into nearly nothing. Sort the projection on a different column, and NUMERIC(19) jumps to 4.03 bytes/row - about 27× larger. NUMERIC(38) lands in essentially the same place as NUMERIC(19) in this regime (4.03 vs 4.03 bytes/row), because the bytes representing the values themselves are similar; the extra slot width just compresses out the same way it does in either case.

Performance under unfavorable sort follows a different pattern from UC2. Scan ratio survives at parity - both n18 and n19 read the same ~1 byte/row of physical storage even when unsorted (n18 doesn't have much to compress regardless, and n19's full slot width gets reduced by AUTO to roughly the same level). GROUP BY and JOIN, however, become much more expensive: n19 costs 2.7× the GROUP BY time of n18 and 3.2× the JOIN time. INSERT remains a clean 1.3× regardless of sort order - this turns out to be one of the most reassuring findings in the whole investigation, because it means migration timing isn't penalized further by sort layout.

We verified one more thing that's worth surfacing: putting the typed column second in a multi-column ORDER BY (id, k) gives identical bytes/row to ORDER BY id alone - 4.029 either way. AUTO encoding is essentially binary; there is no partial credit for putting the typed column near the front of a compound sort. Either it's the primary key for the projection, or it isn't.

Recommendation

When the typed column can't be the projection's primary sort, plan for the unfavorable numbers: about 4× storage, 2.7× GROUP BY, 3.2× JOIN. INSERT cost ratio is unchanged. The single most effective optimization is to add a dedicated projection sorted on the typed column for queries that filter or aggregate by it, while keeping the main projection sorted by your dominant access pattern. That's standard Vertica projection design - it just becomes a high-leverage decision once you know the sort-order penalty.

One scale note. The sort-order penalty grows with row count: 10.85× at 500M rows, 26.79× at 2B rows. We did not measure beyond 2B, but the trend is super-linear, and large production tables can be expected to see meaningful additional storage delta at scale. The performance ratios are more stable across scale (we saw drift within ±5% between 500M and 2B), but the storage gap is the one to plan for.

Use Case 4 - "Should I use FLOAT instead of NUMERIC?"

This is the question every Vertica practitioner has been asked at least once. It usually arrives either as a performance argument ("NUMERIC is slow") or a precision argument ("I only need 6 decimals anyway"). The answer depends entirely on what range of values you need to represent, not on speed.

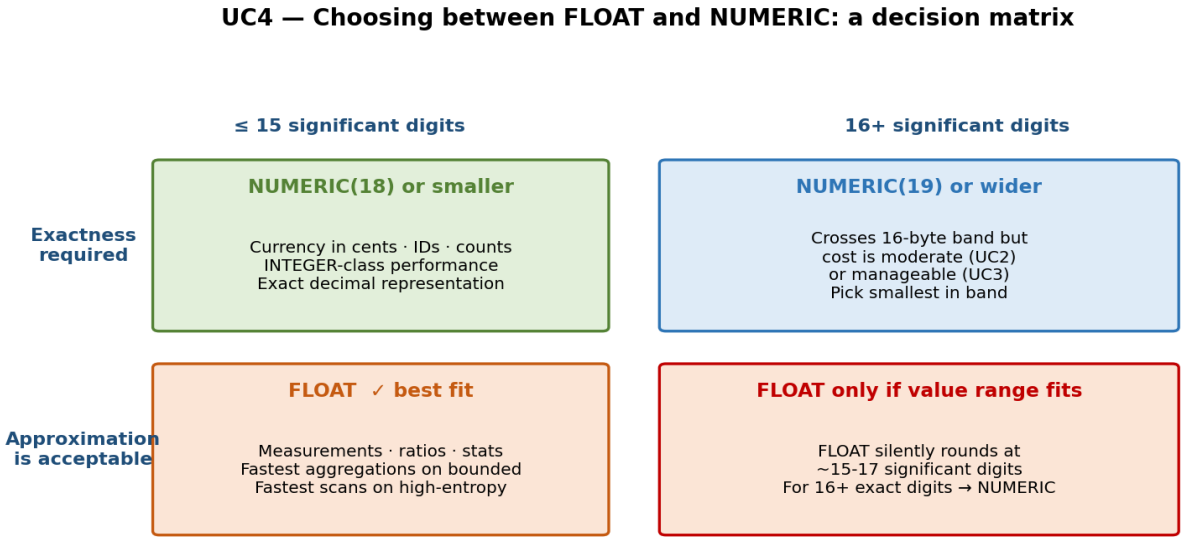

Figure 4 - A two-dimensional decision matrix: exactness requirement on the vertical axis, value range on the horizontal.

FLOAT in Vertica is IEEE 754 double precision: about 15–17 significant decimal digits, fixed at 8 bytes regardless of magnitude. Round-tripping a 19-digit value through FLOAT loses information silently - 1234567890123456789 comes back as 1234567890123456768. A million rows of 0.01 stored as FLOAT and summed produces 9999.99999999988 instead of an exact 10000.00. These aren't Vertica-specific quirks; they're how IEEE 754 has worked since 1985. They become the wrong tool the moment exactness matters.

On the other hand, FLOAT genuinely shines for legitimate measurement workloads - sensor readings, ratios, statistical quantities, anything where the input was already approximate. The 8-byte fixed width compresses well under AUTO encoding, hardware floating-point arithmetic is fast on modern CPUs, and the analytical operations (averages, standard deviations, regressions) tolerate and even expect the precision FLOAT offers. UC5 covers this case in detail.

Recommendation

Choose by exactness, then by range. If equality compares matter or the value represents currency / IDs / counts, use NUMERIC. If you're modeling measurements or statistics and the magnitude fits in ~15 significant digits, FLOAT is the right tool and may be meaningfully faster on aggregations. Outside those guardrails - exact behavior with 16+ digits - NUMERIC(19+) is the best answer.

Use Case 5 - "I'm storing approximate measurements at scale"

Sensor readings, telemetry, prices, statistical measurements: workloads where each value was already approximate when it arrived. FLOAT is the natural fit, but how does it actually perform compared to NUMERIC on the kind of high-entropy data these workloads produce?

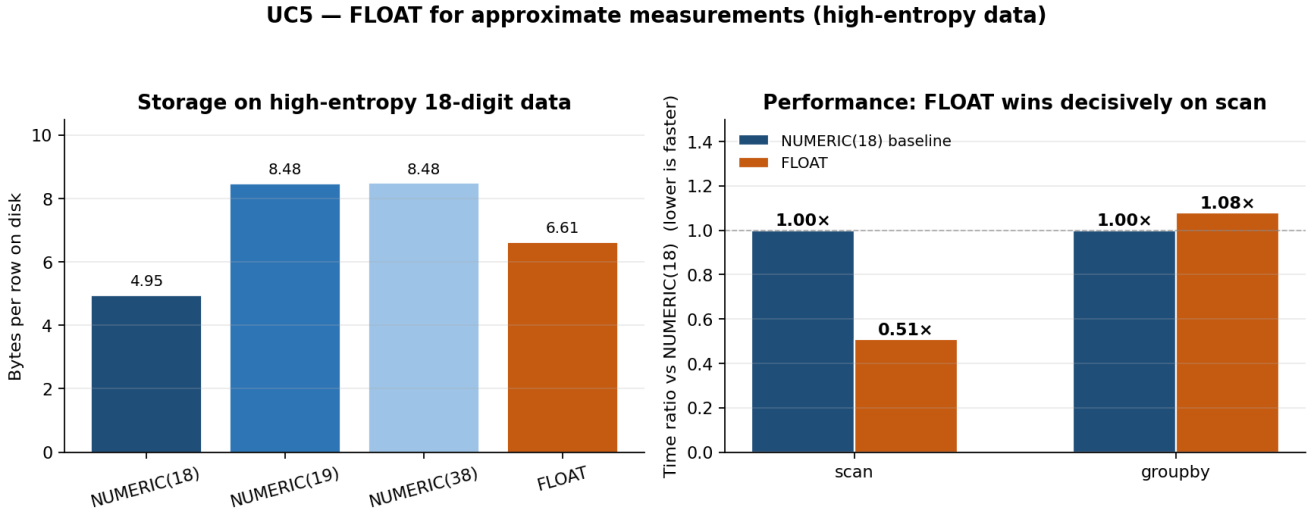

Figure 5 - On high-entropy 18-digit data, FLOAT cuts scan time roughly in half versus NUMERIC(18). GROUP BY is essentially a wash.

Two patterns from the data. Storage: FLOAT lands at 6.61 bytes/row on high-entropy values, between NUMERIC(18)'s 4.95 and NUMERIC(19)'s 8.48. The 8-byte fixed width gets compressed somewhat (the exponent bytes are often constant across nearby values) but not as aggressively as either NUMERIC. Performance is the more interesting story: FLOAT scan is 0.51× the time of NUMERIC(18), thanks to hardware floating-point and the smaller per-row footprint. GROUP BY is 1.08× - essentially the same as NUMERIC(18). The difference is most visible on read-heavy aggregation pipelines, which is exactly where measurement workloads tend to spend their time.

There is one more consideration that doesn't fit on the chart. FLOAT's lossy nature is a feature for measurement data, not a bug. Sensor readings rarely need bit-for-bit fidelity, statistical aggregations actively prefer the smoothing that floating-point arithmetic introduces, and downstream visualization tools generally consume FLOAT or even FLOAT4 anyway. Storing as NUMERIC would be paying for an exactness guarantee that no downstream consumer can use.

Recommendation

For genuinely approximate measurement data - sensors, prices that don't need to balance to the cent, ratios, statistics - FLOAT is the right type. You get faster scans, comparable GROUP BY, and the storage overhead vs NUMERIC(18) is modest. Reach for FLOAT confidently when both the input and the consumer of the output are tolerant of ~15-digit precision.

Use Case 6 - "My AVG is silently rounding large numbers"

Vertica's built-in AVG returns FLOAT (double precision). For most analytical workloads - averaging timestamps, prices, ratios - that's fine and fast. The catch arrives when the values themselves push past about 16 significant digits, at which point FLOAT silently rounds the result. We measured the cliff precisely.

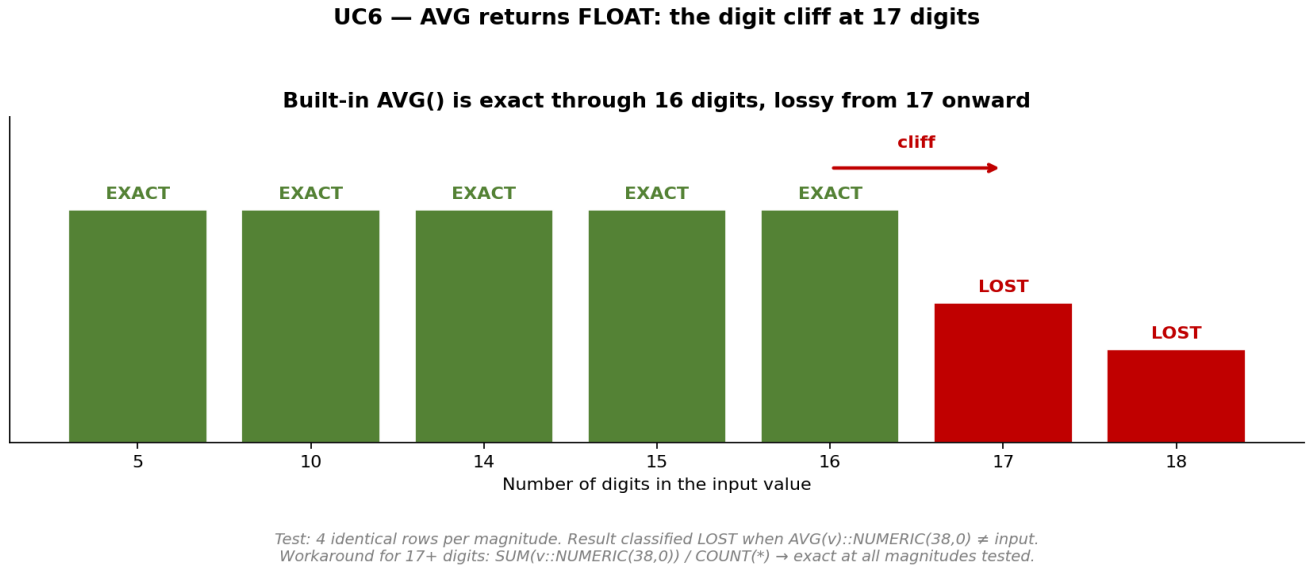

Figure 6 - AVG returns exact results through 16 digits and rounds at 17. The test compares AVG(v)::NUMERIC(38,0) directly against the input value.

Test setup: four identical rows per magnitude, so AVG should equal the input. At 14, 15, and 16 digits, AVG returns exactly the input value - the difference column is zero.
At 17 digits, AVG returns 12345678901234568 when the input is 12345678901234567 - a one-digit error.
At 18 digits, AVG returns 1e+18 (= 1,000,000,000,000,000,000) when the input is 999,999,999,999,999,999 - the result is silently rounded up across the next power of ten.

The cleanest workaround is a manual SUM-divided-by-COUNT, casting the operand inside the SUM:

```
SELECT SUM(v::NUMERIC(38,0)) / COUNT(*) AS exact_avg
FROM   big_table;
```

This returned the exact NUMERIC value at all five magnitudes we tested. The pattern keeps NUMERIC arithmetic end-to-end; the result is a NUMERIC, not a FLOAT. One small caveat: division in Vertica adds 18 digits of scale to the operand's precision, so the SUM can't be wider than NUMERIC(1006) or the division will fail - NUMERIC(38) is the practical sweet spot.

For the rare workload that needs even more headroom - extremely high precision combined with extremely large row counts - we maintain two open-source UDxes that keep the math exact end-to-end: exact_sum() at github.com/mogomo/vertica-exact-sum-udx and exact_avg() at github.com/mogomo/vertica-exact-avg-udx. They dynamically extend the working precision based on input precision and row count and raise a clear error rather than returning silently rounded results when even NUMERIC(1024) is insufficient. Both repositories include production-use disclaimers and are best validated in your own environment before adoption.

Recommendation

For most analytical workloads, built-in AVG() is the right tool - fast, simple, and exact through 16 digits. When the result might exceed 16 significant digits and exactness matters, switch to SUM(v::NUMERIC(38,0)) / COUNT(*). For extreme precision-times-volume workloads, consider the exact_avg() UDx.

Use Case 7 - "It's currency. What do I do?"

Currency is the canonical exactness-required workload. Pennies must balance, bookkeeping audits must reconcile, and equality comparisons must work the way humans expect. NUMERIC is the right type, full stop - FLOAT's silent rounding is dangerous in this context. The remaining question is whether 18 digits is enough, and what to do about SUMs at scale.

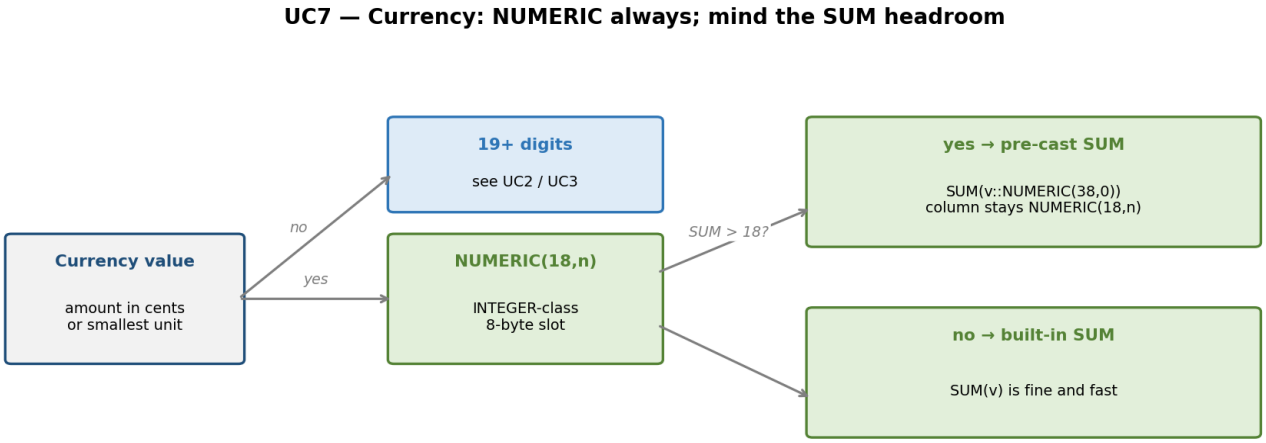

Figure 7 - Currency decision tree. NUMERIC always; choose precision based on individual value range; pre-cast the SUM if aggregations exceed 18 digits.

For most retail and financial workloads, individual transaction values fit comfortably in NUMERIC(18,2) (or NUMERIC(18,0) when storing in the smallest unit, like cents or satoshis). That keeps the column in the 8-byte band with INTEGER-class performance. The wrinkle is aggregation: a year's worth of transactions across millions of accounts can produce SUMs that easily exceed 18 digits, and as we saw in UC1, default Vertica configuration may silently wrap that overflow. The fix is the pre-cast pattern we covered in UC1 - no migration needed, and the column itself stays in the 8-byte band where the dominant per-row operations remain fast.

If individual values themselves exceed 18 digits - large institutional transfers, certain cryptocurrencies' raw amounts, derivative position notional values - you're in UC2 or UC3 territory. NUMERIC(28) buys nine digits of headroom for free at the runtime level, and the projection design choices in UC2 versus UC3 determine your storage cost.

Recommendation

Store currency as NUMERIC, never FLOAT. For typical workloads NUMERIC(18,2) or NUMERIC(18,0) (storing in the smallest unit) is exactly right; pre-cast inside large SUMs to handle aggregation overflow without changing the column type. For values that themselves exceed 18 digits, NUMERIC(28) is the recommended target - see UC2 / UC3 for storage and performance implications.

Use Case 8 - "I need to migrate an existing column across the precision band"

So you've decided the column needs to grow past NUMERIC(18). The natural first attempt is the obvious one. It doesn't work.

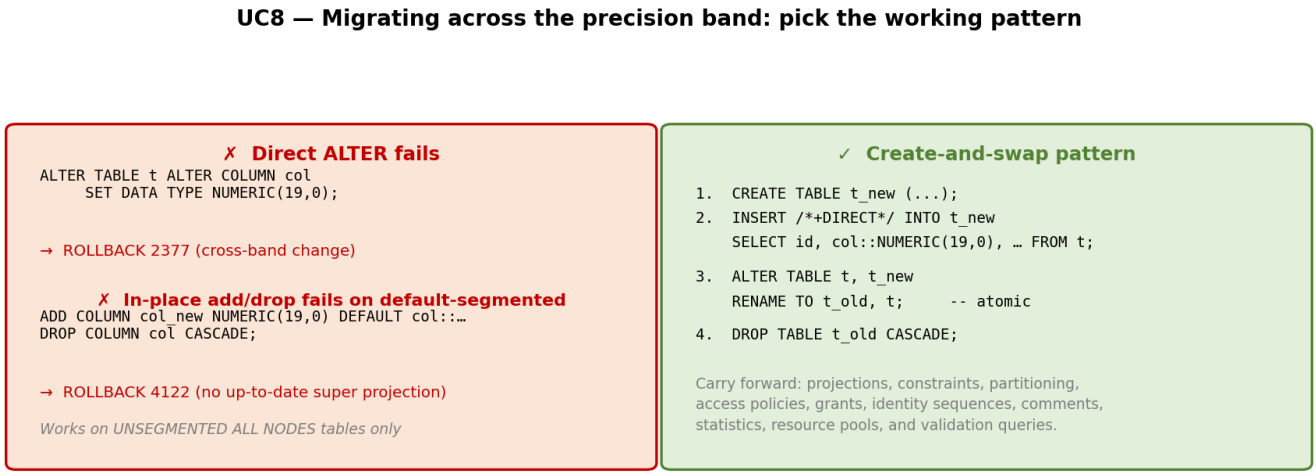

Figure 8 - Direct ALTER fails across the precision band; the create-and-swap pattern is the working alternative for default-segmented k-safe tables.

Vertica refuses ALTER TABLE ... SET DATA TYPE NUMERIC(19,0) on a NUMERIC(18,0) column with ROLLBACK 2377: the change crosses an 8-byte storage boundary, which Vertica classifies as a storage reorganization rather than a metadata change. The protection is sensible - it stops you from accidentally rewriting a petabyte-scale column with a one-line statement - but it means you need a different approach.

The documented in-place swap pattern (ADD COLUMN, populate via DEFAULT, DROP COLUMN, RENAME) does work, but only for tables you can fully control. On a default-segmented k-safe table - the standard production layout with _b0/_b1 buddy projections - the DROP COLUMN step returns ROLLBACK 4122: "No up-to-date super projection left" because the buddy projections still reference the original column. The hint in the error message is the right hint: drop the table, or replace the projections first. We tested SELECT REFRESH(...), explicit DO_TM_TASK('mergeout', ...), and replacement-projection workflows; on a default-segmented table they were not sufficient because the originals still exist and Vertica correctly refuses to drop a column they need.

Two situations where in-place swap does work cleanly: UNSEGMENTED ALL NODES tables (small reference and dimension tables, replicated identically per node, no buddy projections); and tables where you've already manually replaced every projection that references the column. For most production fact tables, either of those workflows is comparable in effort to the create-and-swap pattern below - which is why we recommend create-and-swap as the default.
The create-and-swap pattern in four steps:

```
-- 1. Build the replacement table with the target type
CREATE TABLE schema.t_new (
    id   INT,
    col  NUMERIC(28,0)              -- the only change
    -- ... copy every other column, projection, constraint exactly ...
);

-- 2. Copy data with the explicit cast
INSERT /*+DIRECT*/ INTO schema.t_new
SELECT id, col::NUMERIC(28,0)       -- and other columns
FROM   schema.t;
```

```
COMMIT;

-- 3. Atomic multi-table rename
ALTER TABLE schema.t, schema.t_new
      RENAME TO t_old, t;

-- 4. After verification, drop the old
DROP TABLE schema.t_old CASCADE;
```

The multi-table rename is a single metadata operation. Throughout steps 1 and 2, the original t is fully readable on its original name; the only moment of cutover is the rename itself. If something goes wrong before the rename, you simply drop t_new - the original is untouched.

A short checklist of things INSERT … SELECT won't carry into the new table, and that you'll want to script from the original DDL before you start:

- Every projection (column lists, ORDER BY, SEGMENTED BY, encodings)
- Every constraint (PK, FK, NOT NULL, DEFAULT, CHECK)
- Partitioning
- Access policies
- Grants and ownership
- Identity columns and sequences (last value, ownership, dependencies)
- Comments on the table and on individual columns
- Statistics (ANALYZE_STATISTICS on the new table after load and again after rename)
- Resource pool assignment and any tuning specific to the table
- Pre-rename and post-rename validation queries (row counts, checksums, NULL counts on key columns) and a documented rollback path back to t_old

Recommendation

For default-segmented k-safe production tables, use create-and-swap. For UNSEGMENTED ALL NODES tables, in-place add/drop/rename works cleanly. In either case, scripting the surrounding DDL (projections, constraints, grants, statistics) is more work than the data move itself - plan accordingly, validate before the rename, and keep t_old around long enough to be confident.

Putting it together: a one-page decision framework

Walking the eight use cases in order is one way to use this guide. Walking the framework below is faster when you already know what you're looking at.

Step 1 - Exactness or approximation? If exact decimal representation matters (currency, IDs, equality compares), use NUMERIC. If the data is already approximate (measurements, ratios, statistics) and fits in ~15 significant digits, FLOAT is faster and storage-efficient.

Step 2 - Does any value exceed 18 digits? No → NUMERIC(18,n) and stay in the 8-byte band. Yes → step 3.

Step 3 - Will the SUM exceed 18 digits even though individual values don't? Apply the pre-cast pattern (UC1). No column migration needed.

Step 4 - Crossing the band: how is the projection sorted? Sorted on the typed column (UC2): expect ~1.7× storage on high-entropy data and 1.1–1.3× on hashing operations. Sorted on something else (UC3): expect ~4× storage and 2.7–3.2× on GROUP BY/JOIN; INSERT ratio holds at ~1.3×. Pick NUMERIC(28) for headroom - it costs the same as NUMERIC(19) at runtime.

Step 5 - Will any AVG exceed ~16 digits and need exactness? Switch to SUM(v::NUMERIC(38,0)) / COUNT(*). The exact_avg() UDx covers the extreme tail.

Step 6 - Migrating an existing column? Default-segmented k-safe → create-and-swap. UNSEGMENTED ALL NODES → in-place. Either way, scripting the surrounding DDL is most of the work.

Quick reference for the entire numeric neighborhood

Type	Slot / on-disk	Speed class	Exact?	Where it shines
INT	8 B	Fastest exact integer path	Yes	Surrogate keys, counts, anything integer-shaped
NUMERIC(≤18, s)	8 B / ~1–5 B	INTEGER-class	Yes	Currency in cents, IDs needing decimals, anywhere 18 digits suffices
NUMERIC(19–37, s)	16 B / ~0.15–4 B	1.1–1.3× n18 sorted; 2.7–3.2× unsorted on GROUP BY/JOIN	Yes	19+ digit values, SUM-at-scale safety; pick the smallest precision in the band that comfortably fits
NUMERIC(38–1024, s)	24+ B in 8 B steps	1.2–1.5× n18 sorted; similar to n19 unsorted	Yes	Reach for >37 digits; max precision is 1024
FLOAT	8 B / ~3.4–6.6 B	Faster than NUMERIC(18) on scan; comparable on GROUP BY	No - ~15–17 significant digits, IEEE 754	Measurements, sensors, ratios, statistics
VARCHAR of digits	Variable, large	Slowest by orders of magnitude	Yes (as text)	Almost never for numeric data

Closing thoughts

Eight scenarios, eight recommendations, one underlying lesson: in Vertica, the choice of numeric type is genuinely consequential, but it is rarely the most consequential decision you make about that column. Projection sort order matters more than precision (UC2 vs UC3). The aggregation pattern matters more than the column type (UC1, UC6). The exactness requirement matters more than the speed argument (UC4, UC5). And the migration mechanics, once you know the pattern, are a few hours of careful DDL rather than a multi-day project (UC8).

None of this is a knock on Vertica. Quite the opposite - every behavior in this guide is the engine doing exactly what its design promises: AUTO encoding compresses what it can, the 8-byte band stays fast, the create-and-swap pattern protects you from accidentally rewriting a petabyte-scale column. The opportunity is to align your projection design with what AUTO does best and to use the patterns that make Vertica's defaults work in your favor. The cluster will reward you generously when you do.

If you take one thing from this guide, take this: the next time someone asks you whether NUMERIC(18) is enough, the question to push back with isn't "how many digits do you need?" - it's "how is the projection sorted, and what's the SUM going to look like?" The answers to those two questions tell you exactly which use case you're in, and from there the right design follows directly.

About this benchmark

All measurements were taken on a five-node Vertica 26.1.0-0 cluster with approximately 16 GB RAM per node. Each typed table held two billion (id, k, v) rows; smaller targeted tests at 500 million rows confirmed the ratios scale within ±5% over a 4× scale jump.
Per-node row balance was within 0.122% across all eight typed tables. Configuration parameters (AllowNumericOverflow, NumericSumExtraPrecisionDigits) were at documented defaults.
Every query was profiled after CLEAR_CACHES(); reported timings are the median of trailing iterations after dropping the first as warm-up.

The full benchmark script and per-iteration timings are available on GitHub here:
https://github.com/mogomo/vertica-numeric-benchmark